Visión por Computador

Trabajo 2

Contornos

Antonio Gallén, 735184
Alonso del Rincón, 783252

Para ésta práctica se han implementado 3 operadores de detección de bordes: Sobel, Scharr y Canny. De la imagen resultante se determinan las líneas con la implementación de la transformada de Hough que ofrece OpenCV y se han usado para determinar el punto de fuga de una imagen

Para la comparativa de los distintos operadores y sus distintas implementaciones se ha utilizado la imagen aportada de los posters: (*poster.pgm*)

Sobel

A la izquierda la implementación propia, a la derecha el resultado de Sobel de OpenCV

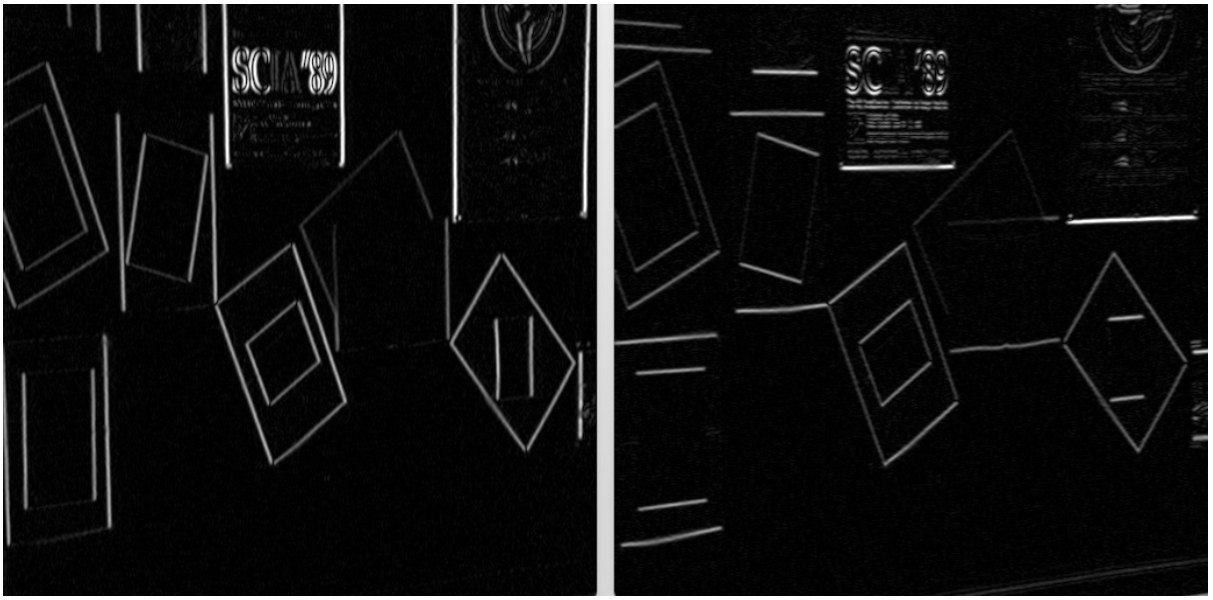
Este operador de detección de contornos trabaja sobre la imagen a analizar tras que se le haya pasado un filtro gaussiano (para reducir el ruido) y una decoloración a escala de grises (para trabajar solo con la intensidad de los píxeles):

La imagen aplicado el filtro gaussiano (es equivalente a la imagen aplicando la decoloración puesto que la imagen original ya viene en escala de grises)

Después de forma convolucional se le aplican dos matrices sobelianas (en este caso de dimensión 3) para detectar los contornos:

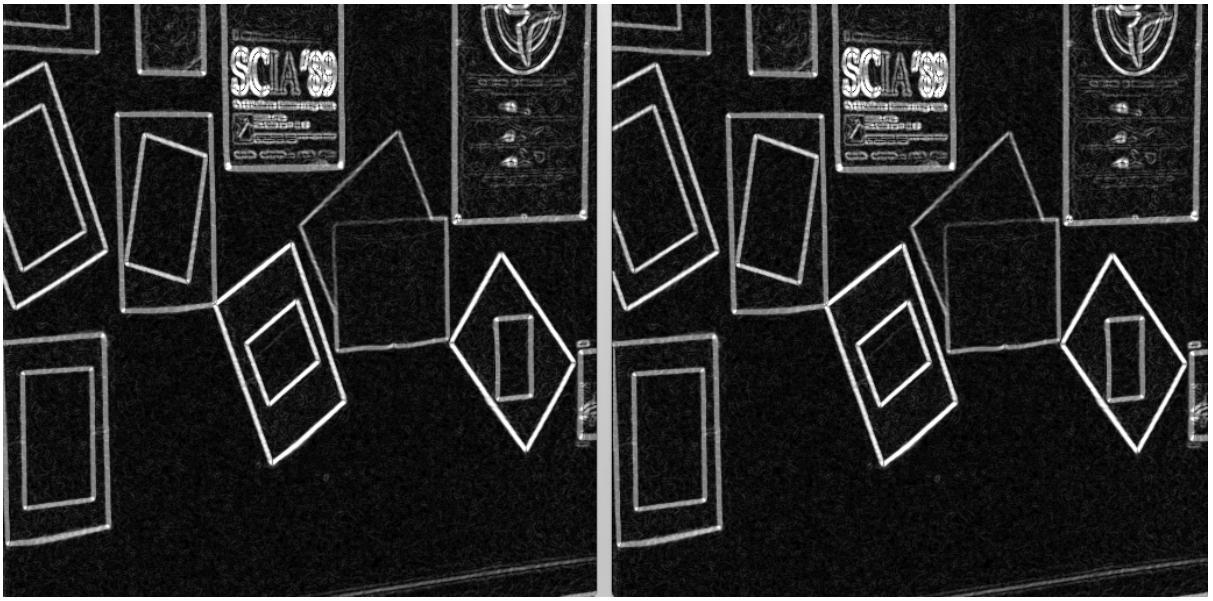
```
Mat horizontalK = (Mat_<float>(3, 3) <<
  1, 2, 1,
  0, 0, 0,
  -1, -2, -1);
Mat verticalK = (Mat_<float>(3, 3) <<
  -1, 0, 1,
  -2, 0, 2,
  -1, 0, 1);
```

Haciendo uso de dichas matrices se detectan los contornos horizontales y los verticales por separado, por lo que es necesario juntar los resultados para que devuelvan obtener la imagen final:



A la izquierda: los contornos verticales; a la derecha: los horizontales

Scharr

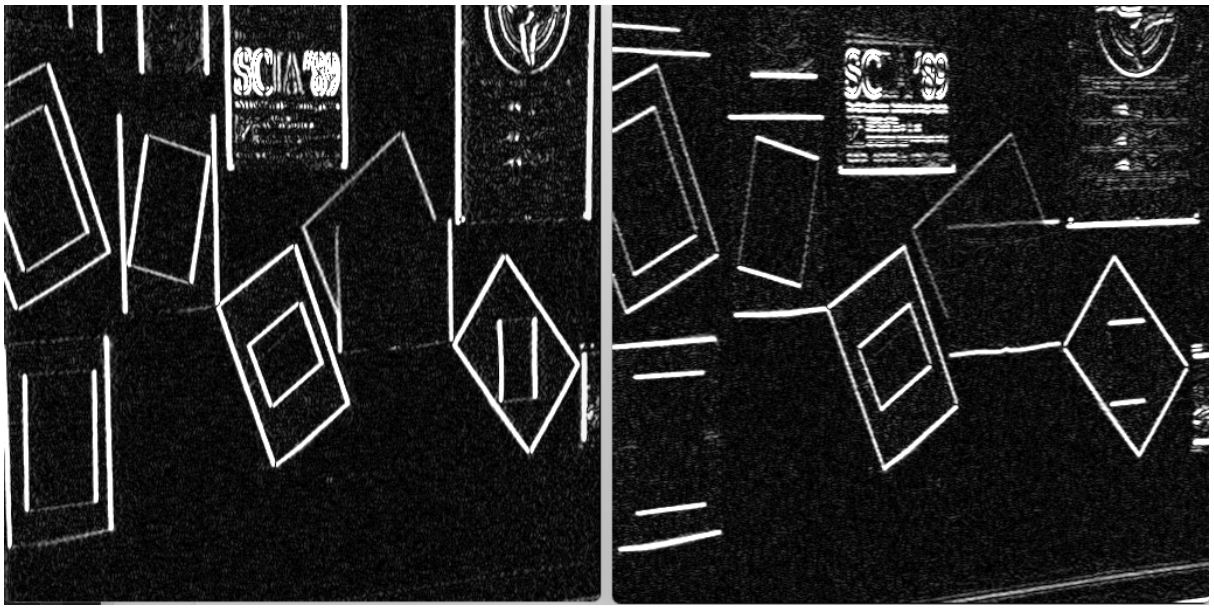


A la izquierda: la implementación propia; a la derecha:la implementación de OpenCV

Este método es idéntico al de Sobel, solo que en este caso las matrices de convolución tienen valores distintos:

```
Mat horizontalK = (Mat_<float>(3, 3) <<
  3, 10, 3,
  0, 0, 0,
  -3, -10, -3);
Mat verticalK = (Mat_<float>(3, 3) <<
  -3, 0, 3,
  -10, 0, 10,
  -3, 0, 3);
```

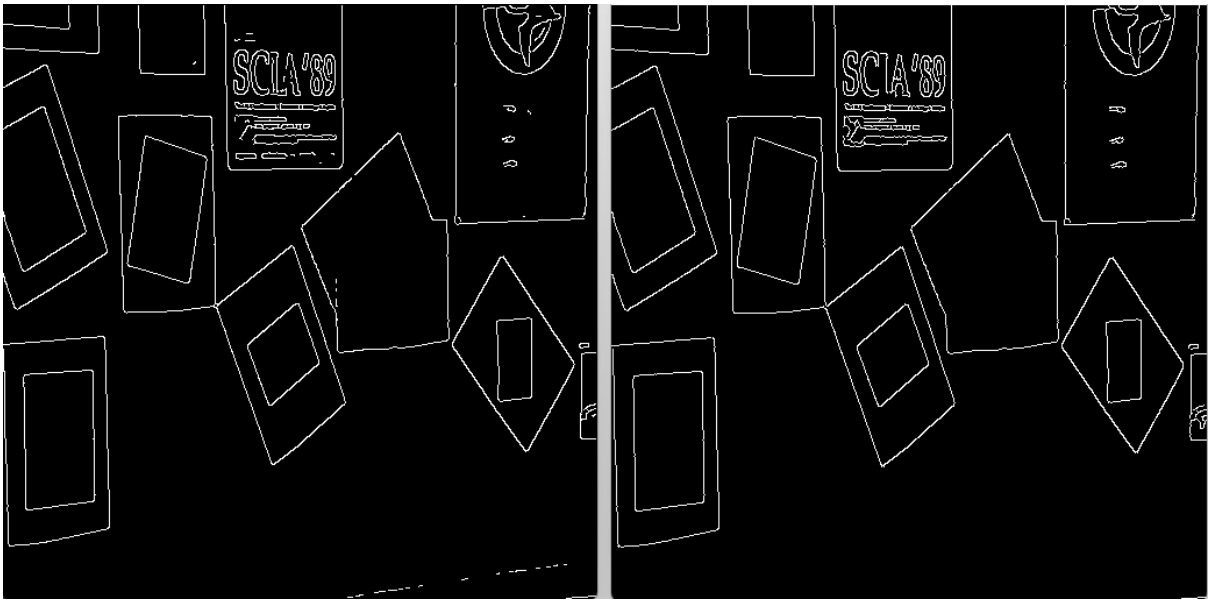
Estas son las imágenes resultantes de los gradientes:



A la izquierda: los contornos verticales; a la derecha: los horizontales

El cambio en la intensidad de los bordes se debe a las diferencias en los kernel utilizados, ya que al ser mayores el resultado es más acentuado (siempre dentro de la propia escala ya que los valores se normalizan)

Canny



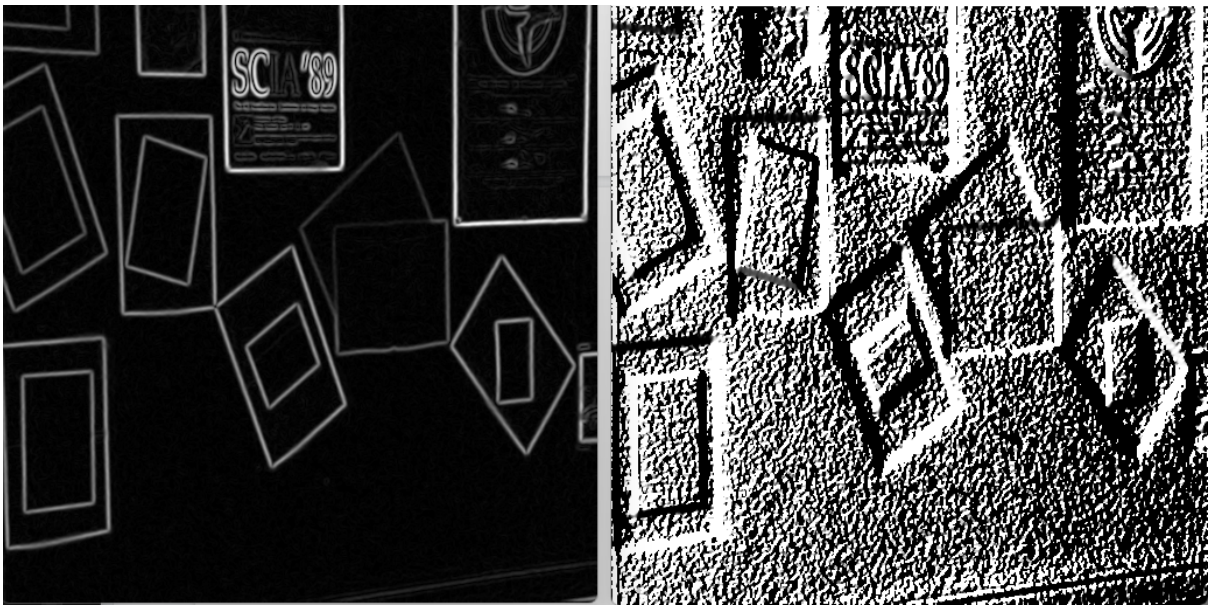
A la izquierda: la implementación propia; a la derecha: la de OpenCV

En esta implementación para detectar los contornos de Canny se han seguido los siguientes pasos:

- Se ha aplicado a la imagen original un filtro gaussiano para reducir su ruido
- Se ha pasado dicha imagen a escala de grises
- Se han calculado los gradientes utilizando las matrices de Sobel
- Con los gradientes por separado (verticales y horizontales, las dos matrices mencionadas con anterioridad) se ha calculado el módulo del gradiente en su conjunto, y su respectivo ángulo del gradiente
- Se ha aplicado una supresión de los valores que no son máximos del módulo del gradiente utilizando los ángulos de dicho gradiente para conseguir “adelgazar” o “estrechar” los contornos detectados
- Se aplica una doble limitación para dividir los píxeles en 2 categorías en función de su módulo del gradiente y se ignoran los que no encajan en ninguna de las categorías
- Se aplica una histéresis en función de la categoría de los píxeles y sus píxeles vecinos para conseguir alargar los contornos

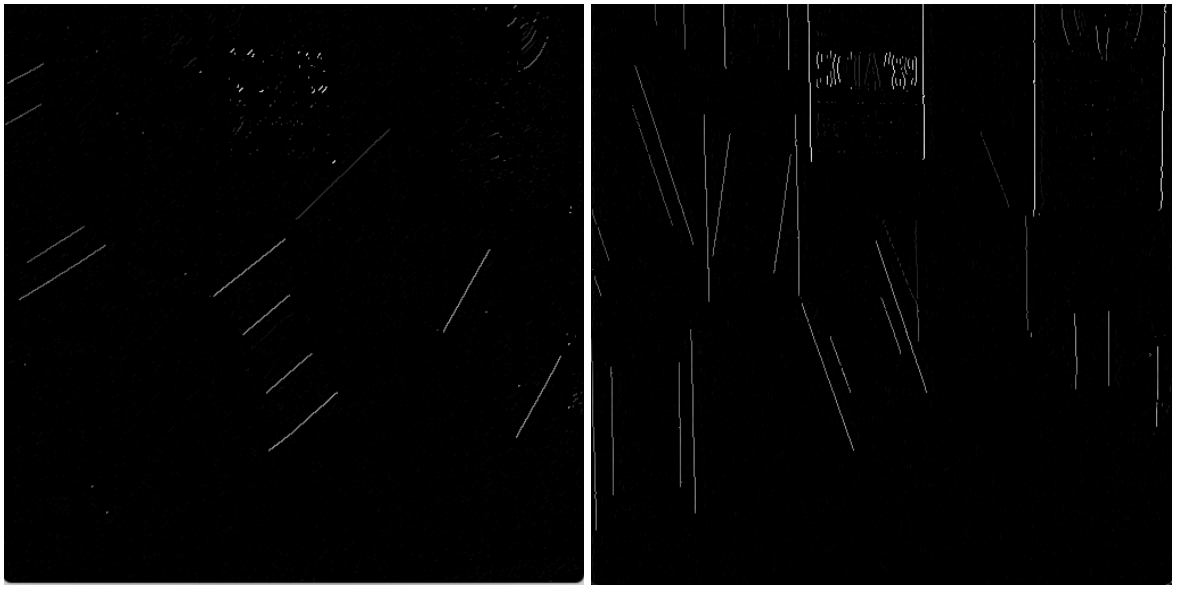
Este proceso visto en más detalle:

Tras aplicar los 3 primeros pasos, y haber sacado las gradientes con las mismas matrices utilizadas en Sobel se obtienen los gradientes horizontales y los verticales, por lo que se procede a calcular con estos el módulo y el ángulo:

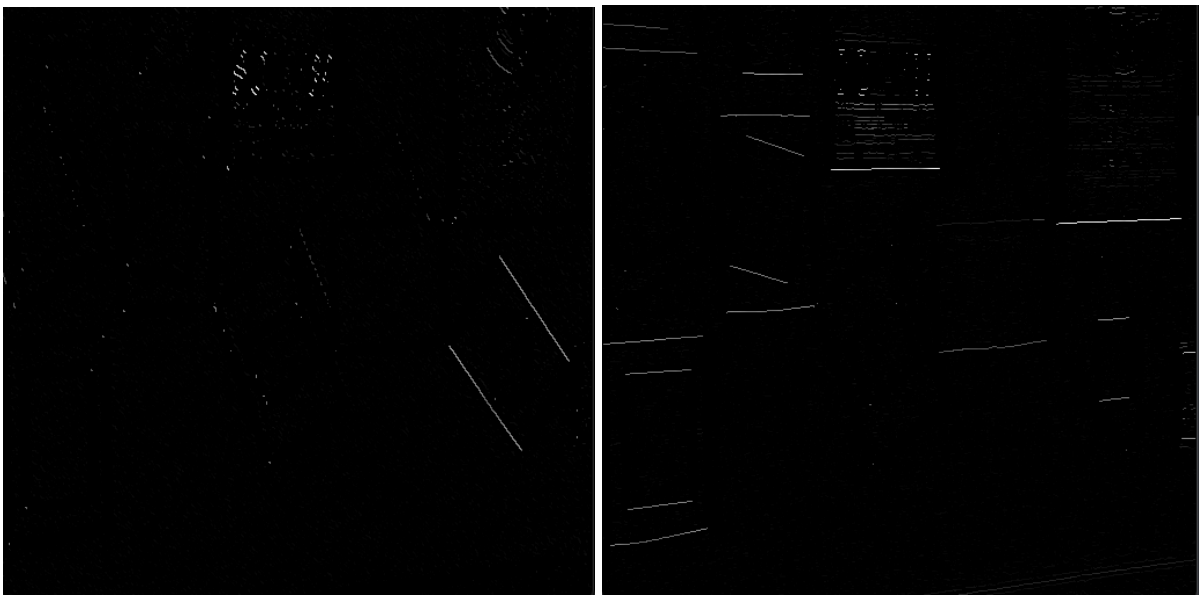


A la izquierda: los módulos resultantes; a la derecha: los ángulos que forman

Para poder hacer más estrechos los contornos para que se puedan trabajar con ellos hay que dejar solo los valores más intensos (ya que se consideran los más centrados en el propio contorno) utilizando la orientación de los módulos. Existen distintas aproximaciones de esta idea que interpolan a más vecinos cercanos pero la idea más sencilla es la siguiente: Aprovechando que los píxeles son cuadrados, y que existen 8 píxeles por definición como mucho que son los más cercanos a cada uno, dos por dirección (véase las direcciones son las formadas por: el píxel norte y el píxel sur, el este y el oeste, el noroeste y el sureste, y el noreste y el suroeste) se aproximan todos los ángulos de los gradientes a las 4 direcciones y se comprueban los máximos en dichas direcciones. Esto es: para todos los píxeles de la imagen, si en su dirección aproximada por el ángulo del gradiente, se comprueba que dicho píxel tiene mayor intensidad que los dos vecinos en su dirección estimada, se deja ese píxel, si no, se actualiza a valor 0:



Componentes de los límites en direcciones noreste-suroeste y norte-sur respectivamente



Componentes de los límites en direcciones noroeste-sureste y este-oeste respectivamente

Resultado conjunto de las 4 componentes (contornos detectados delgados)

El siguiente paso consiste en dividir todos los píxeles en píxeles fuertes y débiles, y descartar el resto (poniéndolos a valor 0). Dicha categorización se realiza en base a dos parámetros que en esta implementación hemos decidido expresar en forma de factores (de 0.0 a 1.0) y que van a dividir los píxeles en función de si la intensidad de cada píxel supera un umbral (pixel débil), los dos (píxel fuerte) o ninguno (píxel descartado)

División de los píxeles en función de su intensidad

Para poder apreciar la división de los píxeles (y para diferenciarlos en el propio algoritmo) en esta imagen (paso intermedio) se ha asignado a cada píxel débil una intensidad de 0.3 y a un píxel fuerte una intensidad de 1.0.

Para finalizar se realiza una iteración de histéresis (podría ajustarse para que se hiciera más de una iteración y no solo una), que consiste en recorrer los píxeles débiles y transformarlos a píxeles fuertes en el caso de tener como vecino al menos a un píxel fuerte, lo cual produce una corrección de los contornos definidos gracias a los píxeles cercanos. Finalmente se devuelve como resultado final la imagen con todos los píxeles a 0 excepto los píxeles detectados al final como fuertes que valen 1

Resultado de la nueva implementación de Canny

Punto de fuga

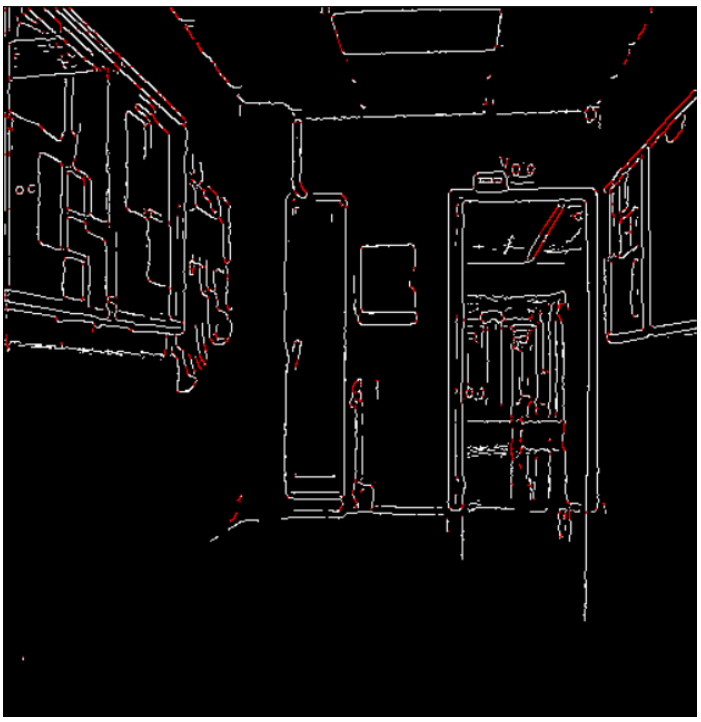
Siguiendo la guía del enunciado se ha considerado la discretización de la línea del horizonte (que se asume la recta y = 0 cuando el centro de coordenadas es el centro de la imagen). Se optó por una discretización en píxeles y no en unidades más grandes (secuencias de píxeles contiguos) a la espera de una fase en el diseño que motivara otra discretización.

Siguiendo el enunciado filtramos con un *threshold* el módulo del gradiente para continuar trabajando solo con *edgels* y, dependiendo del valor de éste, con *edgels* más pronunciados o menos. Éste valor se ha parametrizado pero no se ha llegado a determinar un buen valor para este puesto que el módulo utilizado, el del operador de Canny implementado, ya ha pasado por un proceso de *thresholding* e *hysterisis* por lo que éste valor es redundante para este caso.

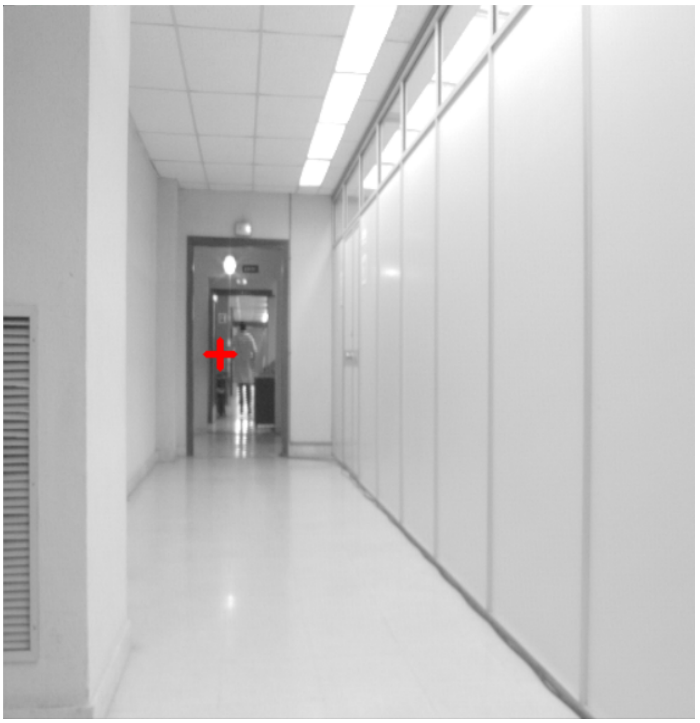
Por la aplicación de la transformada de Hough y siendo que desde el enunciado se obliga a que el ángulo del gradiente intervenga en la votación, cada *edgel* votará por una o varias rectas cuyo parámetro tita (en la parametrización de la recta con rho y tita) está definido por el ángulo del gradiente para ese *edgel*. Se comenzó por una única recta por *edgel* (con tita = la dirección del gradiente en ese píxel) y como el resultado ha sido bueno de ésta forma no se ha considerado aumentar el número de rectas votadas. Las rectas más votadas por los puntos serán las rectas existentes en la imagen. Una posible aproximación a la solución es hacer intersectar dichas rectas con el horizonte y calificar al punto donde más rectas intersectan como el punto de fuga, sin embargo podría ser que las distintas rectas intersectaran en píxeles distintos aunque muy cercanos y el método fallara. En éste punto del diseño no se tomó en cuenta la posibilidad de discretizar más la línea del horizonte por lo que se exploró otra aproximación basada en los siguientes supuestos:

- El punto de fuga es el punto del horizonte donde más rectas existentes en la imagen intersectan (cualquier solución supone esto)
- Si las rectas existentes en la imágen son las más votadas de entre todas las rectas candidatas, la intersección de la mayoría de estas con el horizonte será el punto más votado de éste último.

Siendo así y sabiendo que nuestro objetivo no es encontrar las rectas existentes en la imagen sino el punto de fuga, no se han registrado las rectas votadas por los píxeles sino que tras calcular dichas rectas candidatas se han intersectado con el horizonte y es dicha intersección la que ha recibido el voto.
La existencia de rectas verticales y horizontales o de pendiente muy similar no pasan por el punto de fuga pero sí podrían cortar por el horizonte (siempre en el caso de las verticales) por lo que es conveniente filtrarlas. Solo se ha permitido el voto de rectas cuyo coseno(tita) entre dentro de un rango (un valor cercano a 0 del coseno implicaría una vertical y uno cercano a 1 una horizontal). El rango que mejores resultados ha dado es (0.3 .. 0.7). Ésto puede parecer excesivo y es verdad que veta el voto a pixeles que contribuirían favorablemente a determinar el punto de fuga pero evitan el voto de muchos más píxeles que empeorarían el resultado. En la siguiente imagen pueden verse los *edgels* que han terminado votando:



En rojo los píxeles que han contribuido a la votación.

Los últimos parámetros a ajustar serían los del operador Canny, para los que se han encontrado un sigma de 3, y unos *thresholds* de 0.05 y 0.1 como los mejores valores de entre todos los probados en pasillo2:

Para el caso de pasillo3 los valores de los *thresholds* de Canny han cambiado a 0.1 y 0.3: